



人工智慧與電腦視覺研究室
Artificial Intelligence & Computer Vision Lab

多媒體創新技術研究室
Multimedia Innovative Technology Lab

智慧計算與系統研究室
Intelligent Computing and Systems Lab

Algorithms (Red-Black Tree)

黃祥睿 (Xiang-Rui Huang)

國立澎湖科技大學 資訊工程學系 兼任講師

Adjunct Lecturer, Department of Computer Science and Information Engineering

National Penghu University of Science and Technology Penghu, Taiwan

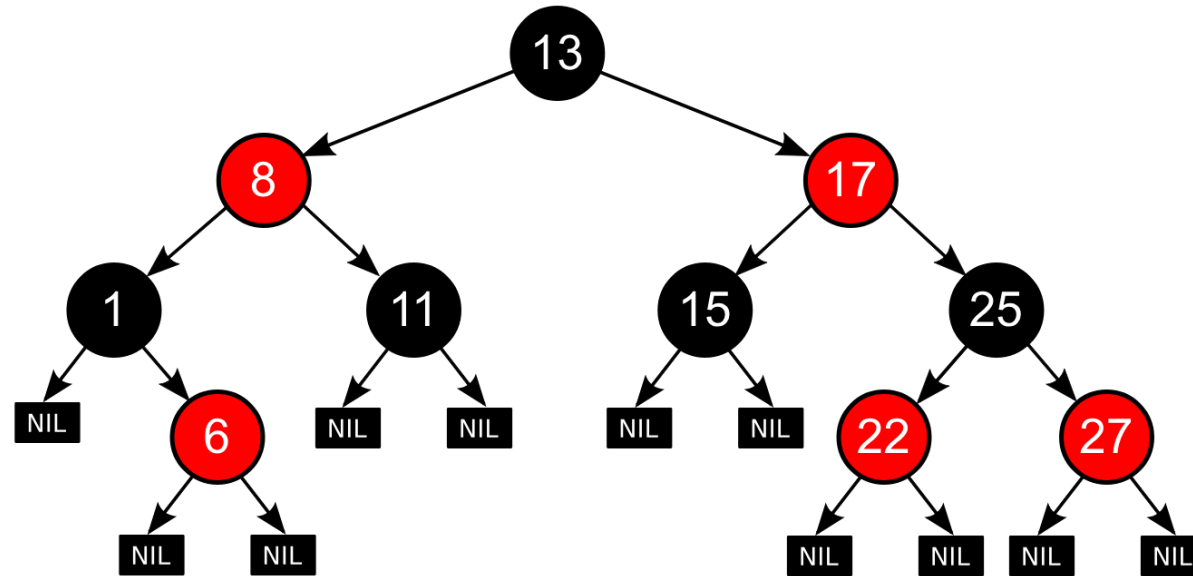
國立陽明交通大學 智慧科學暨綠能學院 博士研究生 (Ph.D. student)

College of Artificial Intelligence, National Yang Ming Chiao Tung University



演算法

Red-Black Tree



<https://brilliant.org/wiki/red-black-tree/>

Red-Black Tree

紅黑樹（Red-Black Tree）是一種自平衡的二元搜尋樹（Self-Balancing Binary Search Tree），目的是讓查找、插入、刪除都能維持在 $O(\log n)$ ，避免一般 BST 因為節點插入順序不佳而退化成鏈結串列。

紅黑樹的核心是：除了 **key**、**left**、**right** 之外，每個節點都多了一個 **color** 屬性，顏色只有兩種，**Red**（紅）與 **Black**（黑）。透過顏色限制來強制整棵樹維持平衡。

建構紅黑樹時必須遵守五個重要原則：

1. 每個節點不是紅色就是黑色。
2. 根節點（**Root**）一定是黑色。
3. 所有葉節點（**Leaf**，也就是 **NIL / NULL** 虛擬節點）都必須是黑色。這裡的葉節點不是一般理解的「沒有小孩的節點」，而是每個 **NULL** 指標都視為一個黑色 **NIL** 節點。
4. 紅色節點不能有紅色的小孩，也就是不能出現 **Red $\rightarrow$ Red** 的情況，這稱為 **No Red-Red Violation**。只能是 **Red $\rightarrow$ Black**。
5. 從任一節點到所有後代 **NIL** 節點的路徑上，黑色節點的數量必須相同，這稱為 **Black Height**（黑高一致）。這條規則保證樹不會某一邊特別深。

因為有這些限制，所以紅黑樹可以保證：最長路徑長度不會超過最短路徑的 2 倍。原因是紅色節點不能連續出現，最多只能黑紅黑紅交錯，因此樹高可以控制在 $h \leq 2\log(n+1)$ ，所以搜尋、插入、刪除的時間複雜度都能維持 $O(\log n)$ 。

Red-Black Tree

插入新節點時的流程如下：

第一步，先完全按照普通 **BST**（**B**inary **S**earch **T**ree）的方式插入，也就是遵守左小右大的規則，先不管顏色。

第二步，新插入的節點一律先設為紅色（**Red**）。原因是如果一開始插入黑色，會直接破壞黑高一致（**Rule 5**）；而插入紅色只可能破壞紅紅相連（**Rule 4**），比較容易修正。

第三步，檢查是否出現違規，最主要是檢查是否出現「紅色父節點 + 紅色子節點」的 **Red-Red Violation**。

第四步，如果違規，就使用 **Rotation**（旋轉）與 **Recoloring**（重新染色）來修復平衡。

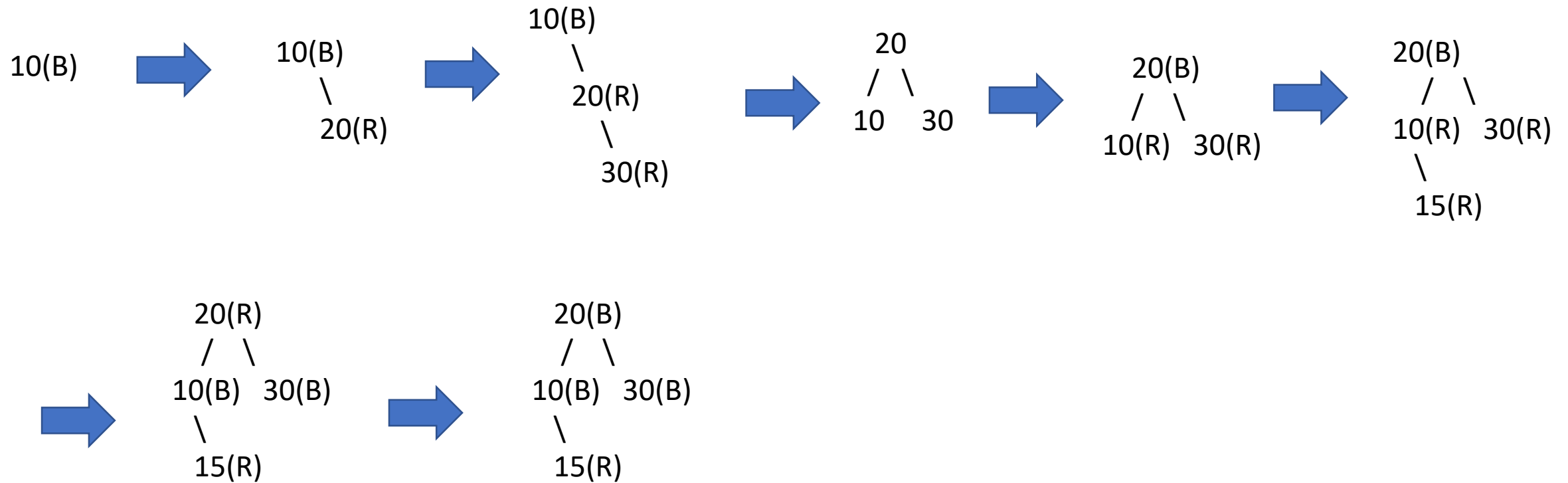
修復主要有兩種手段：

第一種是 **Recolor**（重新染色）。例如當父節點是紅色、叔叔節點也是紅色時，通常會將父節點改成黑色、叔叔節點改成黑色、祖父節點改成紅色，然後把問題往上繼續處理。

第二種是 **Rotation**（旋轉），分成 **Left Rotation**（左旋）與 **Right Rotation**（右旋）。常見有四種失衡型態：LL、RR、LR、RL，與 **AVL Tree** 的概念很相似。

Red-Black Tree

10 , 20 , 30 , 15



Assignment 1 (2026/4/27)

Assignment 1 – Red-Black Tree

Assignment 1 : Red-Black Tree

- Implement all details of the class with code.
- Printing time complexity



人工智慧與電腦視覺研究室
Artificial Intelligence & Computer Vision Lab

多媒體創新技術研究室
Multimedia Innovative Technology Lab

智慧計算與系統研究室
Intelligent Computing and Systems Lab

Algorithms (Hamming Distance)

黃祥睿 (Xiang-Rui Huang)

國立澎湖科技大學 資訊工程學系 兼任講師

Adjunct Lecturer, Department of Computer Science and Information Engineering

National Penghu University of Science and Technology Penghu, Taiwan

國立陽明交通大學 智慧科學暨綠能學院 博士研究生 (Ph.D. student)

College of Artificial Intelligence, National Yang Ming Chiao Tung University



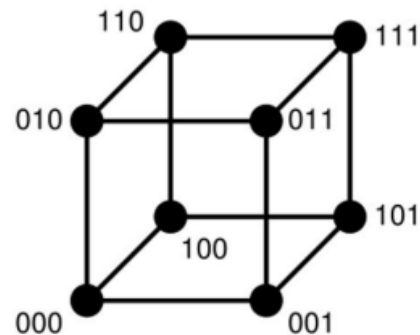
演算法

Hamming Distance

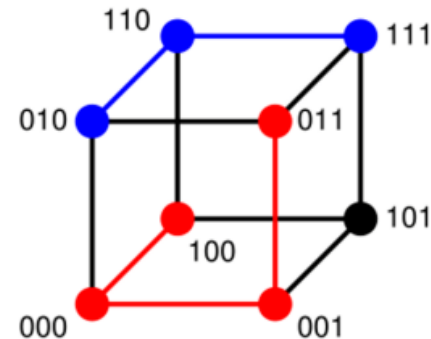
(Binary and categorical data)

- Number of different attribute values.
- Distance of (1011101) and (1001001) is 2.
- Distance (2143896) and (2233796)
- Distance between (toned) and (roses) is 3.

3-bit binary cube



100→011 has distance 3 (red path)
010→111 has distance 2 (blue path)



Assignment 2 (2026/4/27)

Assignment 2 – Hamming Distance

Assignment 2: Hamming Distance

- Implement all details of the class with code.
- Printing time complexity

Assignment 3 (2026/4/27)



人工智慧與電腦視覺研究室
Artificial Intelligence & Computer Vision Lab

多媒體創新技術研究室
Multimedia Innovative Technology Lab

智慧計算與系統研究室
Intelligent Computing and Systems Lab

Algorithms (Integral Image)

黃祥睿 (Xiang-Rui Huang)

國立澎湖科技大學 資訊工程學系 兼任講師

Adjunct Lecturer, Department of Computer Science and Information Engineering

National Penghu University of Science and Technology Penghu, Taiwan

國立陽明交通大學 智慧科學暨綠能學院 博士研究生 (Ph.D. student)

College of Artificial Intelligence, National Yang Ming Chiao Tung University



演算法

Integral Image

1	2	2	4	1
3	4	1	5	2
2	3	3	2	4
4	1	5	4	6
6	3	2	1	3

input image

0	0	0	0	0	0
0	1	3	5	9	10
0	4	10	13	22	25
0	6	15	21	32	39
0	10	20	31	46	59
0	16	29	42	58	74

integral image

Assignment 3 – Integral Image

Assignment 2: Integral Image

- Implement all details of the class with code.
- Printing time complexity